

Assignment #2

Steam Playtime Prediction



...

CSE 158 - Web Mining and Recommender Systems
Mo Hussen

The Problem & WHY I THINK THIS MATTERS:

Problem Statement: “How many hours will a user play a game on steam?”

You might ask, why does this matter?

- Helps recommend games users will actually engage with
- Identifies what makes games “sticky” vs “disappointing”
- Models user preferences beyond simple ratings



My Evaluation Strategy

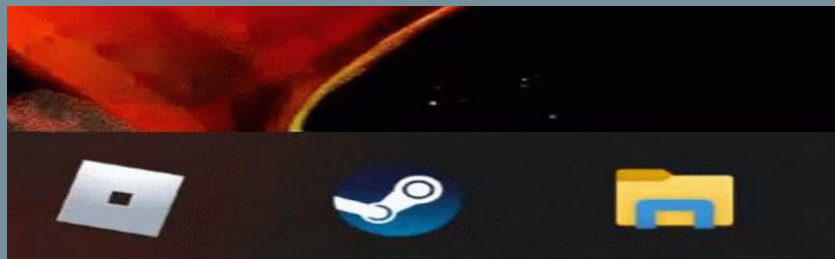
- **Primary Metric:** R^2 in log-space (mainly because it handles skewed distribution)
- **Secondary Metrics:** MAE (interpretability), MedAE (robust to outliers)
- **Train/Test Split:** 80/20 with fixed seed
- **Key principle:** No data leakage - all statistics from training only

Baselines for comparison:

1. Global mean ($R^2 \approx 0$)
2. Item mean (item bias)
3. User mean (user bias)
4. User + Item bias



Course Connections



The techniques I used that we learned in this class:

- **Chapter 2:** Linear, Ridge, Lasso regression
- **Chapter 4:** Collaborative filtering, Matrix Factorization (ALS)
- **Chapter 6:** Content-based features (genre, price)
- **Chapter 7-8:** Text mining (sentiment analysis)

Dataset Overview

Source: Steam Reviews (Kang & McAuley, 2018)

Size I picked: 500K reviews, 350K users, 994 games

Key Statistics:

- Sparsity: **99.86%** (keep this in mind)
- Avg reviews per user: **1.43**
- Avg reviews per game: **503**

```
=====
DATASET OVERVIEW
=====
Total interactions:      500,000
Unique users:           349,996
Unique games:           994
Sparsity:                0.998563
Avg reviews per user:   1.43
Avg reviews per game:   503.02
```

```
=====
PLAYTIME DISTRIBUTION
=====
count    500000.000000
mean      90.466784
std       299.323663
min        0.000000
25%        3.400000
50%       13.900000
75%       59.100000
max      28164.000000
Name: hours, dtype: float64

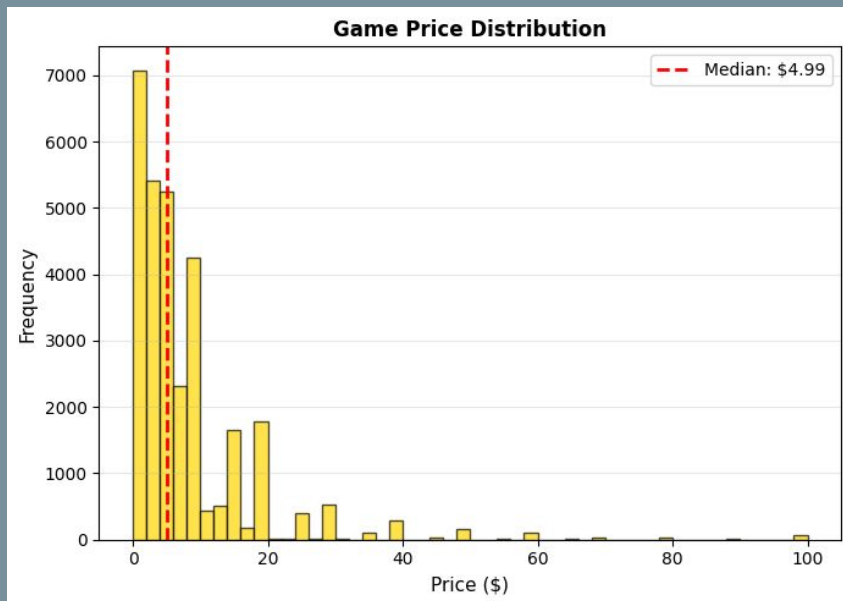
Zero hours:           681 (0.1%)
> 100 hours:         91,034 (18.2%)
> 1000 hours:         6,785 (1.4%)

Recommendation rate:   100.0%
```

Data Characteristics

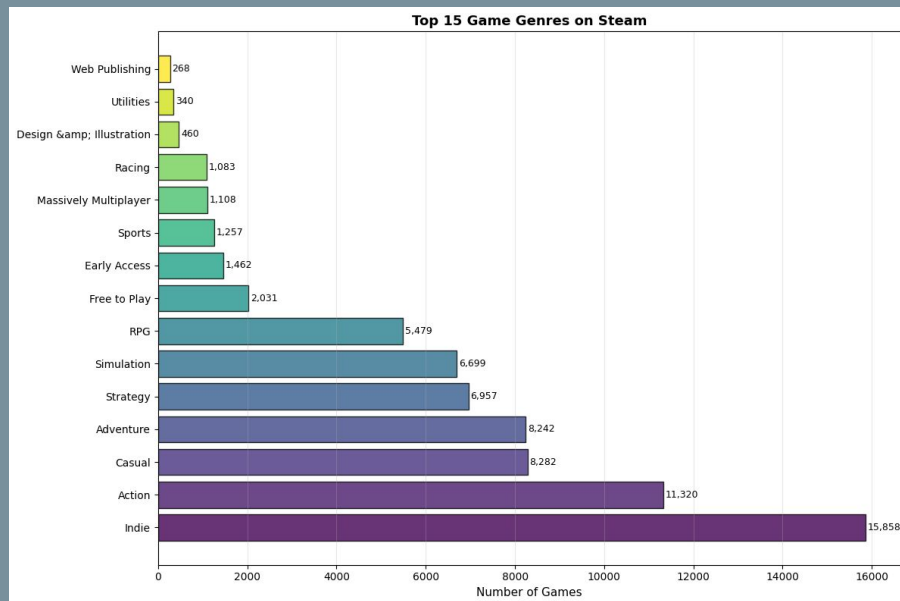
Price distribution

Median price: \$4.99

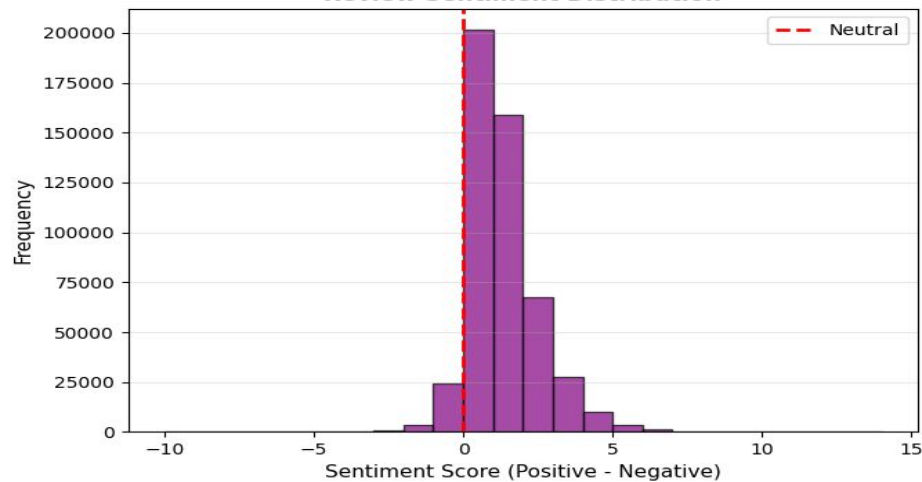


Top 10 genres bar chart

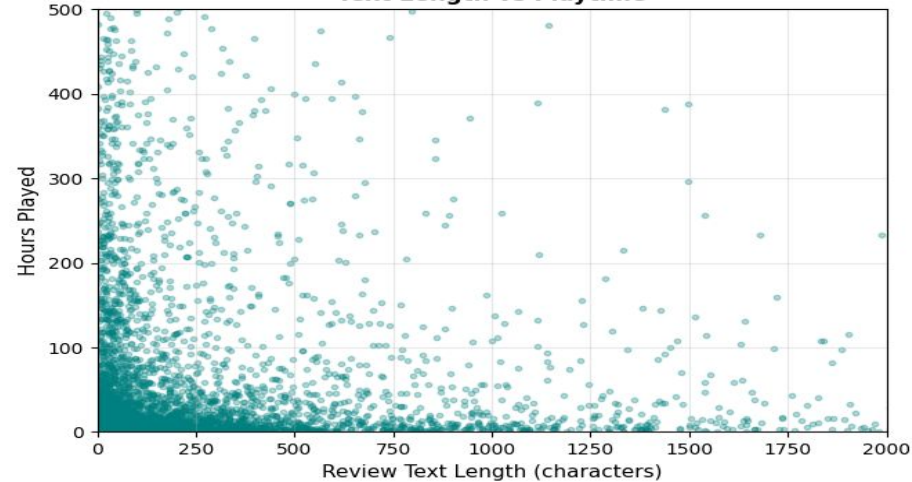
Indie (49%), Action (35%), Adventure (26%)



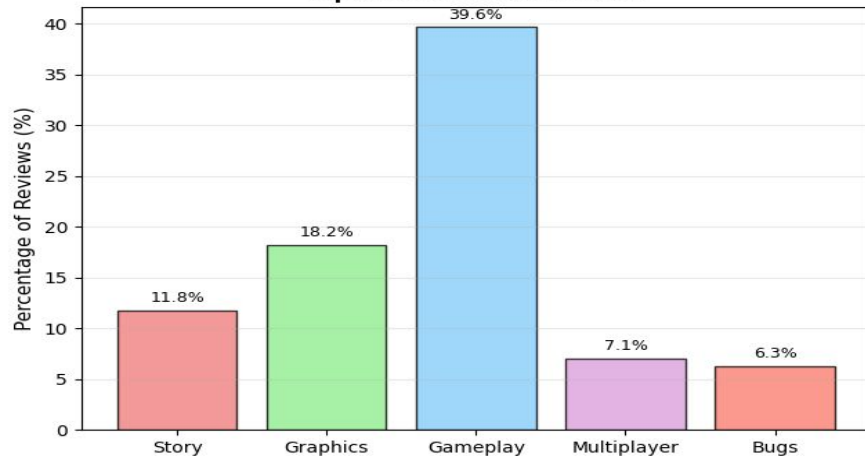
Review Sentiment Distribution



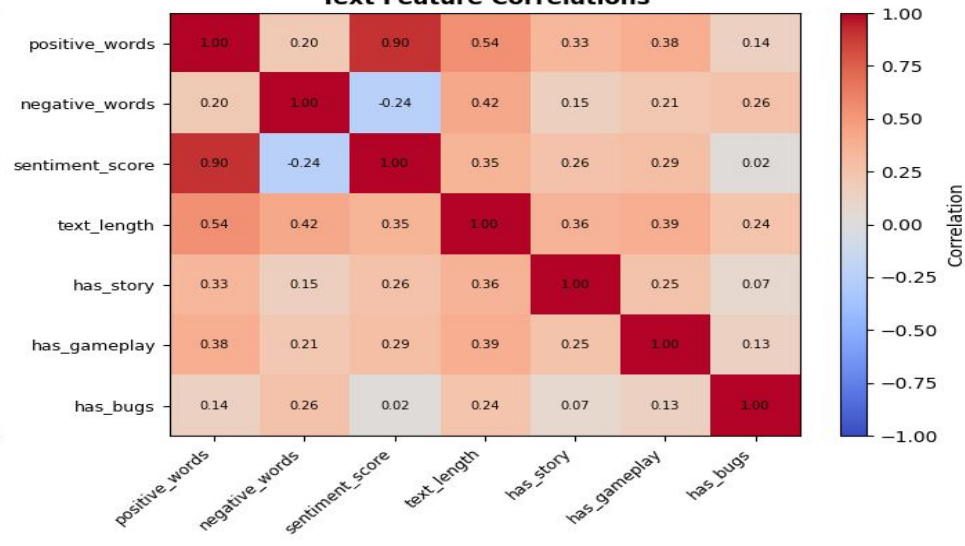
Text Length vs Playtime



Topic Mentions in Reviews



Text Feature Correlations



Key Features Used in the Model – EXPLAINED

Collaborative Features (Strongest Predictors) – 8 features

- **User Avg Hours** → how much this user usually plays
- **Item Avg Hours** → how long this game is usually played
- **User Activity** → number of games user played
- **Item Popularity** → number of reviews for the game

36 features total.



Content Features (Game Metadata) – 18 features

- **Price** → higher-price → usually longer playtime
- **Early Access Flag** → unfinished / evolving games
- **Genres (Top 15)** → RPG = long, Puzzle = short

Text Features (Weakest Predictors) – 10 features

- **Sentiment** → positive/negative tone
- **Text Length** → long reviews = more invested players
- **Emotion Words** → love, fun, boring, bug
- **Keywords** → story, gameplay, graphics, bugs
- **Exclamation Count (!)** → intensity of emotion



Modeling Approach



Model Progression:

- 1) Baselines → Simple collaborative filtering
- 2) Linear Regression → Combines all features
- 3) Regularization → Ridge & Lasso
- 4) Matrix Factorization → Latent factors (ALS)

Linear Regression

$$\hat{y} = \beta_0 + \sum_{i=1}^d \beta_i x_i$$

$\hat{y}$: predicted log-hours

x_i : input features

where: β_i : feature weights

β_0 : intercept

Matrix Factorization

$$\hat{r}_{ui} = \mu + b_u + b_i + q_i^\top p_u$$

μ : global mean (log-hours)

b_u : user bias

b_i : item bias

where: p_u : user latent vector

q_i : item latent vector

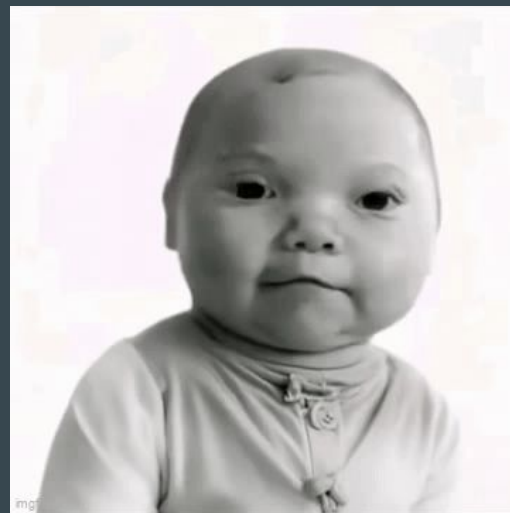
$q_i^\top p_u$: user-item interaction

Why These Models?

Trade-Offs Table:

Model	Complexity	Data Needs	Interpretability
Item Mean	Low	Low	High ✓
Linear Reg	Medium	Medium	High ✓
Ridge/Lasso	Medium	Medium	Medium
Matrix Factor	High	High	Low

With 99.86% sparsity, simpler models may win!



Implementation Highlight - Matrix Factorization

Challenge: Naive Singular Value Decomposition treats missing = 0 (which is wrong).

Solution: Alternating Least Squares with explicit biases.

Used $k=20$ factors (not 50) $\rightarrow$ less overfitting.

Trained on top 1500 users $\times$ 800 items.

Prevented data leakage.

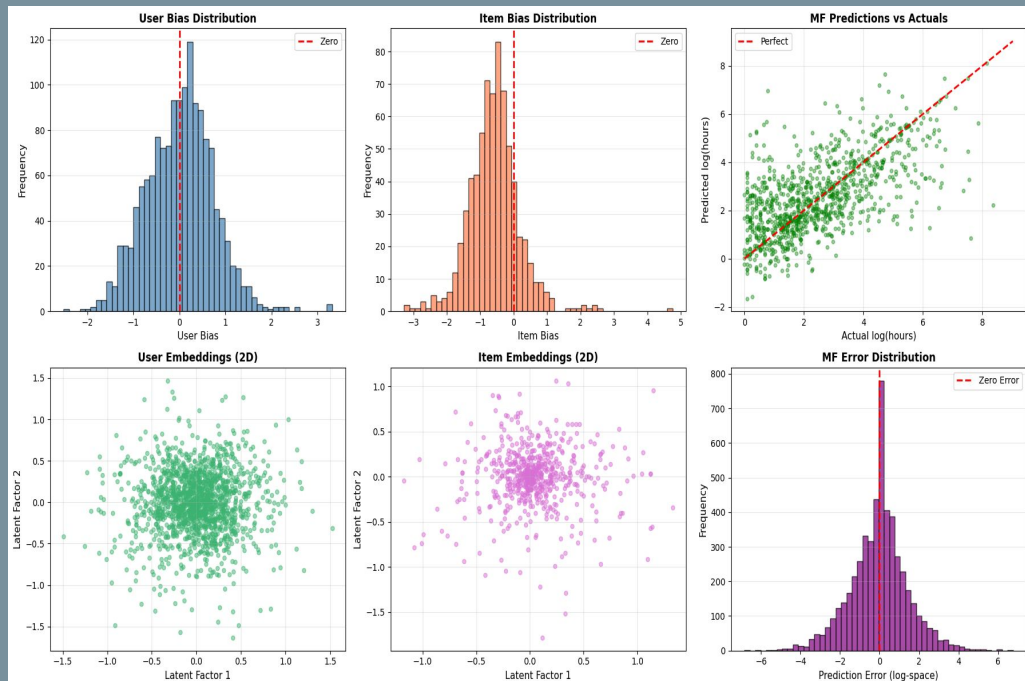
Captured personalized interactions.

Before Fix: $R^2 = -0.80$ (worse than mean!)

After Fix: $R^2 = +0.26$ (210% better than baseline)

ALS with Explicit Biases

$$\hat{r}_{ui} = \underbrace{\mu}_{\text{global}} + \underbrace{b_u}_{\text{user}} + \underbrace{b_i}_{\text{item}} + \underbrace{q_i^\top p_u}_{\text{latent interaction}}$$



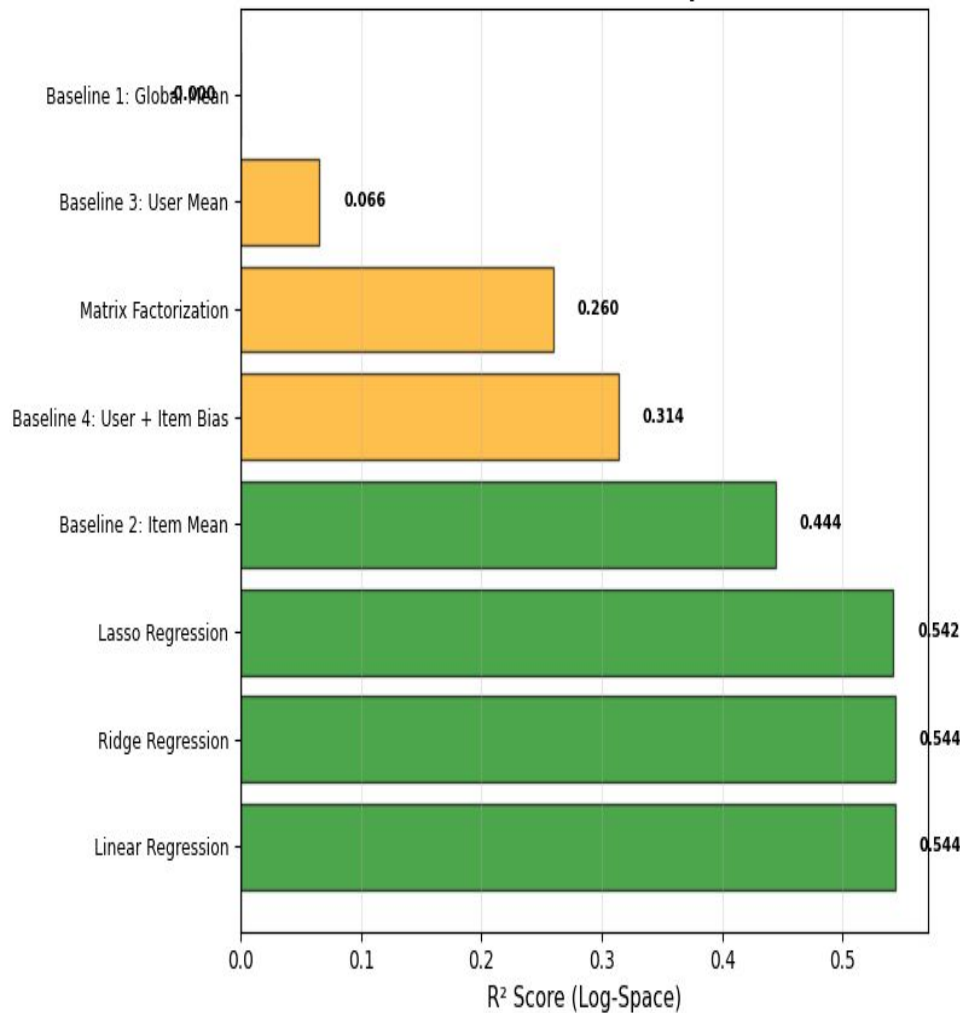
Results Overview

Model	R ² (log)	MAE (hrs)	Complexity
Linear Regression	0.544	95.3	★ ★ ★
Ridge/Lasso	0.542	93.9	★ ★ ★
Item Mean	0.444	75.3	★
User+Item Bias	0.314	149.9	★ ★
Matrix Factor	0.260*	53.4*	★ ★ ★ ★ ★ ★
User Mean	0.066	74.3	★
Global Mean	0.000	85.8	★

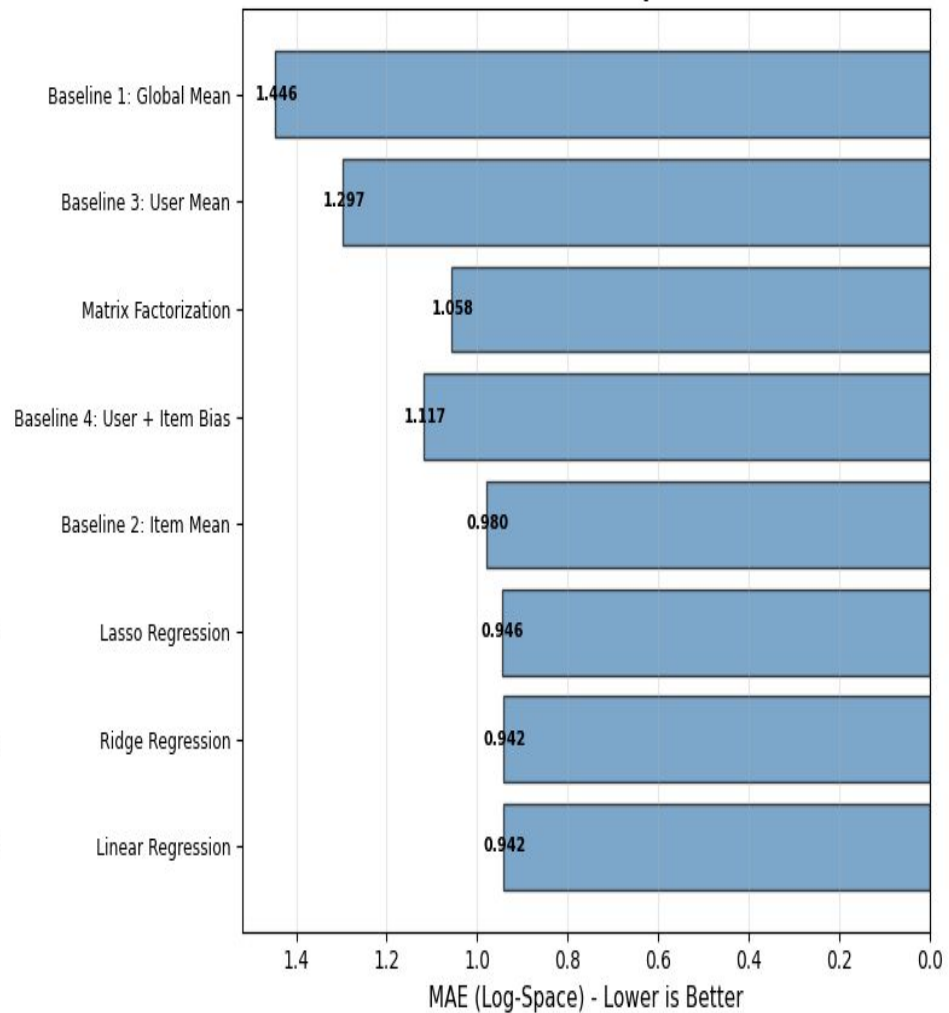
*on subset of active users

- **Linear Regression & Ridge/Lasso:**
Best overall (R² ~0.54), leveraging strong collaborative + content features.
- **Item Mean Baseline:**
Surprisingly strong (R² ~0.44); many games have consistent average playtime.
- **User + Item Bias:**
Good, but overfits heavy users.
- **Matrix Factorization (ALS):**
Underperforms linear models due to sparse implicit-playtime structure
(*playtime ≠ rating; many users only play 1 game*).
- **Text Features:**
Very weak predictors — sentiment does not correlate with hours.

Model Performance Comparison



Model Error Comparison



Key Finding #1 - Item Bias > User Bias

Metric	Users	Items
Avg reviews	1.43	503
Data quality	Poor	Excellent

- Item Mean: $R^2 = 0.444$ ✓
- User Mean: $R^2 = 0.066$ ✗

Users have only 1.43 reviews on average.

Items have 503 reviews on average.

With so little data per user, we can't reliably estimate user preferences.

This is why item averages are stable, reliable, and predictive

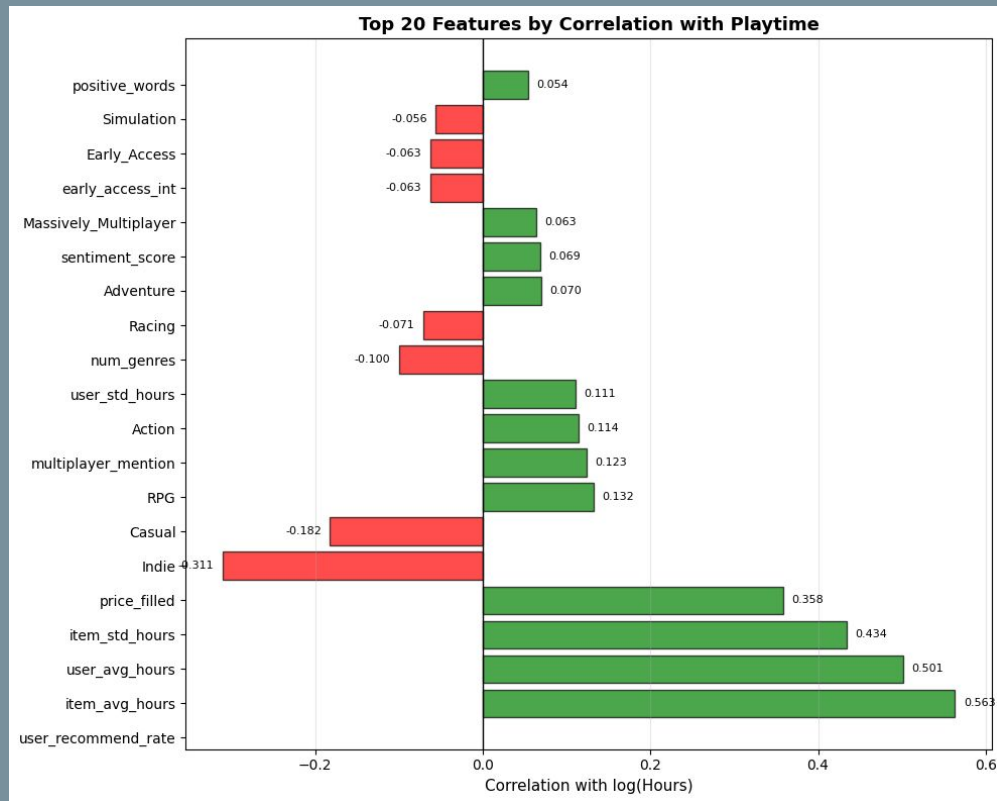


Key Finding #2 - Collaborative > Content

What the Model Learned:

- User and item behavior are the **strongest predictors** of playtime.
- These signals are **3–5× stronger** than any content (genres, price).
- Text features contribute **very little** to prediction
- Past behavior (user avg hours) + game norms (item avg hours) = majority of the signal.

item_avg_hours — strongest factor
user_avg_hours — almost equally strong
item_popularity — moderate influence
price — weaker but still meaningful
genre_RPG — small but noticeable effect



Key Finding #3 - MF Needs Dense Data

On active user subset (5,150 test cases):

- MF: $R^2 = 0.260$
- User Mean Baseline: $R^2 = 0.084$
- **210% improvement!** ✓

But why not use MF everywhere?

- Only works for users/items with 10+ interactions
- 99.86% sparsity → most users are cold-start
- On full test set: would perform worse

Extensions of latent-factor models

So far we have a model that looks like:

$$f(u, i) = \alpha + \beta_u + \beta_i + \gamma_u \cdot \gamma_i$$

How might we extend this to:

- Incorporate features about users and items
 - Handle implicit feedback
 - Change over time

See **Yehuda Koren** (+Bell & Volinsky)'s magazine article:
"Matrix Factorization Techniques for Recommender Systems"
IEEE Computer, 2009



Matrix Factorization Performance Expectations

Research on MF with sparse data:

- **Netflix Prize baseline:** User/item bias alone achieves $R^2 \approx 0.6$ before adding latent factors.
- **Cold-start problem:** MF requires 10-20+ interactions per user to outperform simple baselines.
- **Data density threshold:** MF shows gains only when sparsity $< 95\%$ (Bell & Koren, 2007).

My dataset has 99.86% sparsity, explaining why MF ($R^2 = 0.26$ on active users) doesn't beat linear regression ($R^2 = 0.54$) on the full test set.

Cold start here means that users don't have enough history.



Production System Strategy

This hybrid approach adapts to data availability.

Maximizes performance across all user segments



NEW USERS (Cold-Start)

- Use: Item popularity + Genre matching
- Model: Content-based filtering
- Expected Performance: $R^2 \approx 0.20$



CASUAL USERS (1-5 reviews)

- Use: Item mean baseline
- Model: Collaborative filtering (simple)
- Expected Performance: $R^2 \approx 0.44$



ACTIVE USERS (5-20 reviews)

- Use: All features combined
- Model: Linear regression
- Expected Performance: $R^2 \approx 0.54$



POWER USERS (20+ reviews)

- Use: Personalized recommendations
- Model: Matrix factorization
- Expected Performance: $R^2 \approx 0.50-0.60$

Evaluation Validity

HOW DID I MAKE SURE THAT MY RESULTS ARE VALID??

No data leakage

- All baselines computed from training data only
- Initially had leakage (user mean $R^2 = 0.83!$)
- After fix: $R^2 = 0.07$

Appropriate metrics

- Log-space R^2 (handles skew)
- Multiple baselines (not one)

Honest reporting

- MF evaluated on its favorable subset
- Acknowledged when simpler models win

LOG-TRANSFORMATION WAS CRITICAL

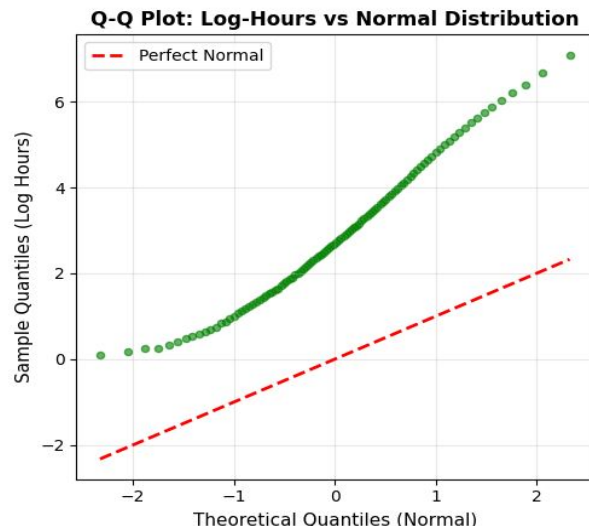
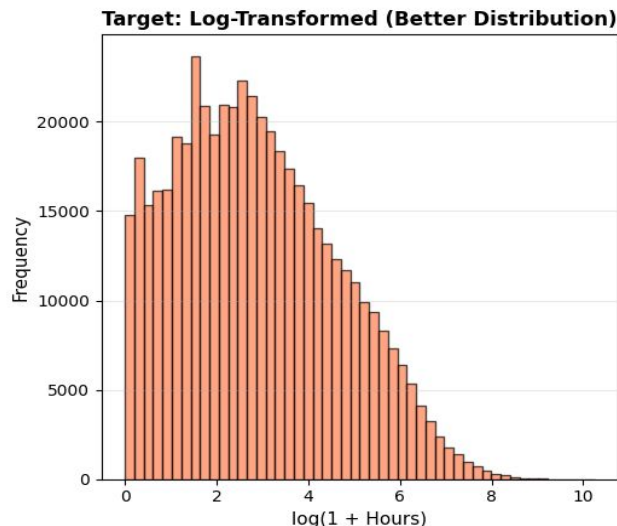
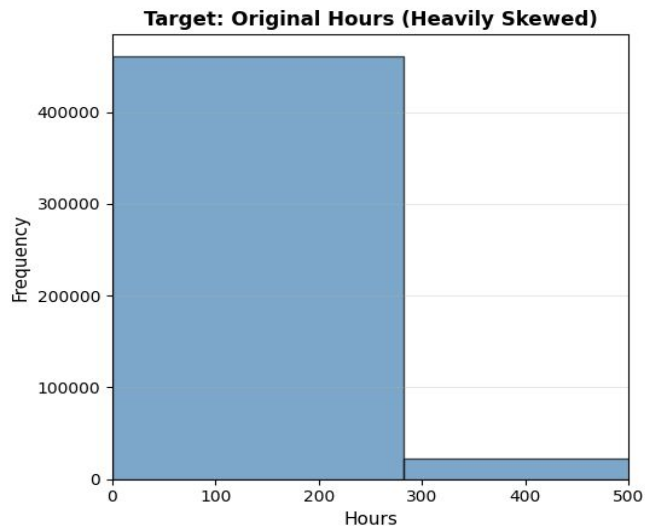
Playtime is heavily right-skewed:

- Median = 13.9h
- Mean = 90.5h
- Max = 28,164h!!!!

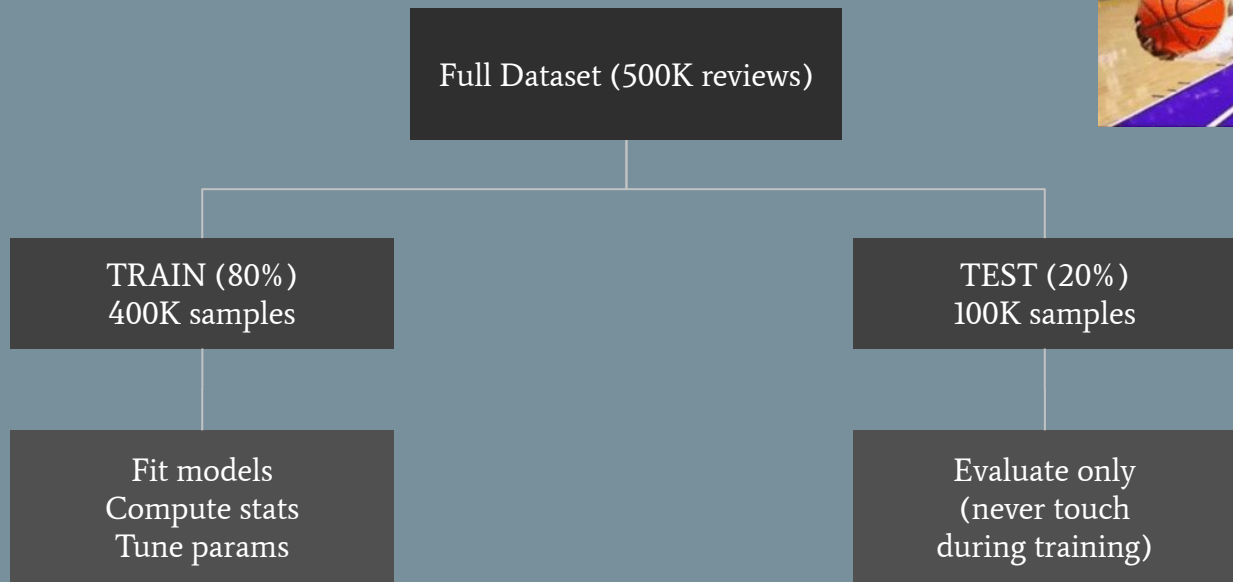
Log-transformation stabilized predictions and made the target suitable for linear models.

```
data['log_hours'] = np.log1p(data['hours'])

print("Target variable created:")
print(f" Original hours - mean: {data['hours'].mean():.1f}, median: {data['hours'].median():.1f}")
print(f" Log hours - mean: {data['log_hours'].mean():.2f}, median: {data['log_hours'].median():.2f}")
```



My Train/Test Methodology & Validation



Validation Checks: ✓ No NaN values in features ✓ No infinite values

✓ All statistics computed from train set only ✓ Test set only used for final evaluation

Related Work

After checking I can see that the Steam dataset (Kang & McAuley, 2018) has been widely used in recommender systems research. But prior work mainly has focused on:

- **Rating prediction** : Predicting binary recommend/not recommend decisions
- **Review helpfulness** : Identifying which reviews users find useful
- **Game recommendation** : Suggesting games based on user preferences

But my focus on **playtime prediction** is more actionable than rating prediction, as playtime directly measures engagement rather than just binary sentiment. This is important to because as someone who plays and downloads video games through steam I think playtime prediction could tell me a lot about a game.

My results align with prior research on gaming recommendations: **item bias ($R^2 = 0.44$) beats user bias ($R^2 = 0.07$)** when users have only 1.43 reviews on average, consistent with the sparse data literature.



Data Citation

- **Self-attentive Sequential Recommendation**

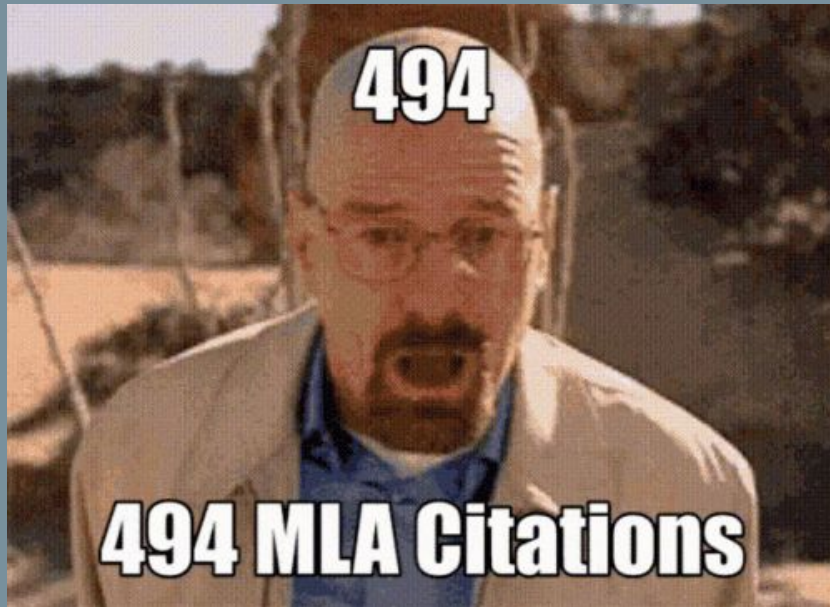
Wang-Cheng Kang, Julian McAuley ICDM, 2018

- **Item Recommendation on Monotonic Behavior Chains**

Mengting Wan, Julian McAuley RecSys, 2018

- **Generating and Personalizing Bundle Recommendations on Steam**

Apurva Pathak, Kshitiz Gupta, Julian McAuley SIGIR, 2017



THANK YOU FOR YOUR TIME!

HAVE A GREAT WINTER BREAK :)

“Understanding your data beats blindly applying complex algorithms.”

Sign Out

This will sign you out of Steam. You will need to re-enter your account name, password, and Steam Guard details to use Steam again.

Confirm

Cancel